

UD 5. Patrón MVC. Framework Laravel

Índice de contenidos

1. Mecanismos de separación de la lógica de negocio.....	3
1.1. Patrón MVC.....	3
1.2. Desarrollo con frameworks.....	5
1.3. ¿Realmente necesito un framework para desarrollar una aplicación web?.....	5
1.4. ¿Qué framework escoger?.....	6
2. Laravel.....	7
2.1. Características de Laravel.....	7
2.1.1. Ventajas de Laravel.....	7
2.1.2. Inconvenientes de Laravel.....	7
2.2. Instalación de Laravel y creación de un proyecto.....	8
2.3. Arquitectura, convenciones y configuración de Laravel.....	10
2.3.1. Estructura de directorios.....	10
2.3.2. Convenciones en Laravel.....	11
2.3.3. Variables de entorno. El archivo .env.....	12
2.4. Artisan, la consola que mola.....	13
2.4.1. Un ejemplo práctico: crear un controlador.....	13
2.4.2. Comandos principales de Artisan.....	14
2.5. Creando la primera página.....	15
2.6. Enrutamiento.....	17
2.6.1. ¿Qué es el enrutador?.....	17
2.6.2. Enrutamiento básico.....	18
2.6.3. Rutas con nombre.....	18
2.7. Controladores.....	19
2.7.1. Cuatro cosas importantes sobre los controladores en Laravel.....	19
2.7.2. Crear un controlador.....	20
2.7.3. Redirecciones.....	20
2.8. Vistas y plantillas. Blade.....	22
2.8.1 Master layout. La plantilla padre.....	22
2.8.2 Vistas que heredan del master layout. Las plantillas hijas.....	24
2.8.3 Cómo pasar variables a las vistas desde el controlador.....	25
2.8.4 Cómo usar las variables desde el controlador en las vistas.....	25
2.8.5 Condiciones en las vistas.....	26
2.8.6 Bucles en las vistas.....	27
2.8.7 Invocación de funciones PHP.....	28
2.8.8 Otras directivas Blade.....	29
2.8.9 Tratamiento de formularios con Blade.....	31
2.8.10 Validación de formularios con Blade.....	32
2.8.11 Añadir recursos (imágenes, css, js...) a nuestras vistas.....	33
2.9. Modelo. ORM Eloquent.....	35
2.9.1 ¿Qué es el mapeo objeto-relacional?.....	35
2.9.2 Configurando los parámetros de la BD.....	35
2.9.3 Crear el modelo.....	36
2.9.4 Añadir campos a la tabla. Migraciones.....	36

2.9.5 Consultas con Eloquent.....	38
2.9.6 Inserciones y borrados con Eloquent.....	39
2.9.7 Inserción de entidades desde un formulario.....	40
2.9.8 Lista de los métodos más útiles de Eloquent.....	40

1. Mecanismos de separación de la lógica de negocio

En informática y ciencias de la computación, el término lógica de negocio (Business Logic) se refiere a aquella parte de un sistema que se encarga de manejar los datos para generar una respuesta a las peticiones del cliente.

Un sitio web típico tiene tres tipos de funcionalidad:

1. El código que implementa la **lógica del negocio**,
2. El código que implementa la **interacción con el usuario** y
3. El código que imprime páginas HTML (**presentación**).

Los programadores usualmente mezclaban estos tres tipos de código en un solo gran archivo PHP que lo hace doloroso de probar y mantener, especialmente cuando escribimos un sitio web grande.

La ingeniería de software ha evolucionado mucho en este sentido, elaborando principios y patrones de desarrollo que nos permitan elaborar un código más estructurado (mejor organizado), modular (componentes independientes intercambiables) y fiable (con menos errores). Uno de esos patrones es el conocido como patrón de diseño **MVC: "Modelo - Vista - Controlador"**.

1.1. Patrón MVC

Existen varios métodos que permiten separar la lógica de presentación (en nuestro caso, la que genera las etiquetas HTML) de la lógica de negocio, donde se implementa la lógica propia de cada aplicación. El patrón MVC pretende dividir el código en tres partes, dedicando cada una a una función definida y diferenciada de las otras.

- **Modelo.** Es el encargado de manejar los **datos propios de la aplicación** y del problema que trata. Debe proveer mecanismos para obtener y modificar la información del mismo, **normalmente utilizando clases**. Si la aplicación utiliza algún tipo de almacenamiento para su información (como un SGBD), tendrá que encargarse de almacenarla y recuperarla.
- **Vista.** Es la parte del modelo que se encarga de la **interacción con el usuario**. En esta parte se encuentra el código necesario para generar el interfaz de usuario (en nuestro caso en HTML), según la información obtenida del modelo.
- **Controlador.** En este módulo se decide **qué se ha de hacer**, en función de las acciones del usuario con su interfaz. Con esta información, interactúa con el **modelo** para indicarle las acciones a realizar y, según el resultado obtenido, envía a la **vista** las instrucciones necesarias para generar el nuevo interfaz.

La gran ventaja de este patrón de programación es que genera código muy estructurado, fácil de comprender y de mantener.

Para saber más ...

En la web puedes encontrar algunos ejemplos de implementación del modelo MVC en PHP. Échale un vistazo al siguiente [artículo](#).

Para entender mejor como funciona esto recordemos que cualquier sitio web solo es un script PHP que recibe una **petición HTTP desde el servidor web y produce una respuesta HTTP**. A continuación se muestra como una petición HTTP es procesada por una aplicación MVC y como se genera la respuesta:

Primero un visitante del sitio web ingresa la URL en su navegador web, y el navegador web envía la petición al servidor web.

1. El servidor web recibe la petición y ejecuta el script de entrada.
2. El script de entrada lo único que hace es crear una instancia de la clase principal de la aplicación.
3. La aplicación llama al componente **enrutador** y le indica la ruta de la petición.

4. El enrutador analiza la ruta y determina a que **controlador** pasar la petición. Si coincide una ruta el controlador es instanciado y se llama al **método de acción** apropiado.

5. Para procesar los datos entrantes el controlador **instancia las clases de modelo** apropiadas y **llama a sus métodos**. Las clases de modelo usa los algoritmos de la lógica de negocio para procesar los datos de entrada y regresar los datos de salida. Los algoritmos de la lógica de negocios son específicos de nuestra aplicación y típicamente incluye recuperar datos de la base de datos, manejar archivos, interactuar con sistemas externos, etc.

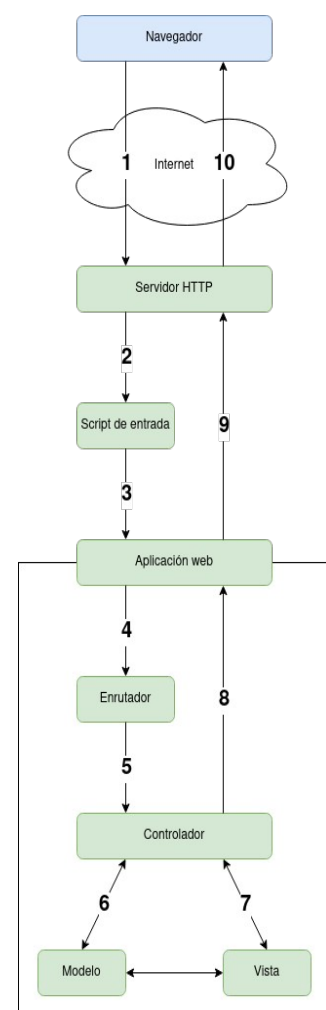
6. Lo que resulta de llamar a los modelos se pasa al **script de vista** correspondiente para generar el código HTML. El script de vista usa los datos que provee el modelo para imprimir la página HTML.

7. El controlador pasa la respuesta HTTP resultante a la aplicación.

8. La aplicación devuelve la respuesta al servidor.

9. El servidor devuelve la respuesta al navegador y este muestra la página al usuario.

Ahora tenemos una idea de como los modelos, las vistas y los controladores cooperan para generar una salida HTML.



1.2. Desarrollo con frameworks

Un **framework** es un conjunto estandarizado de prácticas de programación para resolver una serie de problemas habituales.

El framework proporciona una serie de **clases, librerías y otros componentes** para facilitar el desarrollo ágil, seguro y escalable de nuevas aplicaciones.

Por lo tanto, un **framework MVC PHP** es un conjunto de clases, librerías y otros componentes destinados a servir de base para desarrollar aplicaciones en PHP con arquitectura MVC. La idea es esta: *no inventar la rueda cada vez que la necesitamos*.

1.3. ¿Realmente necesito un framework para desarrollar una aplicación web?

No, los frameworks *nunca* son imprescindibles...

...pero casi siempre son muy convenientes.

Hay programadores pro-frameworks y anti-frameworks. Los programadores anti-frameworks acaban desarrollando, aposta o sin querer, su propio framework después de llevar varias aplicaciones MVC a sus espaldas.

Para decidir si necesitas un framework o no, tienes que conocer antes cómo funcionan y qué ventajas e inconvenientes presentan.

Usar un framework tiene ciertas **ventajas**, como:

- Reutilización del trabajo ya hecho (recuerda: no reinventar la rueda).
- Extensa documentación.
- Separación en capas.
- Seguimiento de buenas prácticas de programación.
- Escalabilidad y mantenimiento.
- Evitar escribir código repetitivo.
- Desarrollo más rápido y, por tanto, más económico.

Pero también tiene algunos **inconvenientes**:

- A veces pueden limitar el desarrollo.
- Curva de aprendizaje costosa (más en unos frameworks que en otros). Es decir: al principio iremos más lentos que sin framework, hasta que le cojamos el tranquillo.
- Puede llegar a implicar más trabajo, dependiendo del proyecto.

- Preferencias personales: algunos programadores solo se sienten cómodos si todo el código es suyo.
- Actualizaciones frecuentes e imprevistas. A menudo, la actualización de un framework hace que parte de tu aplicación deje de funcionar.
- Ocultan gran parte del funcionamiento de la aplicación: no son aptos para aprender a programar.

En resumen: la idea es que, al usar un framework, solo te centras en desarrollar lo importante. El resto (lo que ya estaba desarrollado en el framework y que es común a muchas aplicaciones web) no te quita tiempo.

1.4. ¿Qué framework escoger?

Existen multitud de frameworks MVC PHP. Estos son algunos de gran implantación en la actualidad:

- **Symfony**: el más extendido desde hace años.
- **Laravel**: el que tiene un crecimiento más rápido.
- **CodeIgniter**: el más sencillo, aunque su implantación en la industria el menor.
- Otros frameworks MVC PHP importantes son Zend Framework, Yii y CakePHP.

Lamentablemente, no hay respuesta a la pregunta: ¿qué framework es mejor?

Puedes pasar el resto de tu vida leyendo en foros y en blogs, descargando e instalando varios frameworks para desecharlos a los dos días, esperando a que salga una nueva versión o un framework nuevo...

Nunca podrás responder a esa pregunta. Como mucho, podrás responder a esta otra: ¿qué framework prefiero? Y eso solo después de haber probado varios durante algún tiempo.

Además, el framework por sí sólo puede no ser suficiente para un desarrollo completo, y probablemente harán falta otros componentes que afectarán a la experiencia de programación. Algunos ejemplos son:

- Librerías del lado del servidor: pear, pChart, doctrine...
- Librerías del lado del cliente: jQuery, Vue.js, Angular, React, bootstrap...

En este curso vamos a instalar y utilizar **Laravel**, que es el framework PHP más popular en la actualidad.

2. Laravel

2.1. Características de Laravel

Laravel es un framework PHP MVC para desarrollo rápido de aplicaciones web. Automatiza muchos procesos habituales y tiene una curva de aprendizaje empinada, pero no tanto como otros frameworks (con “otros” queremos decir “Symfony”, que es, sin duda, el framework más difícil de aprender).

Desde hace algunos años, Laravel ha experimentado un crecimiento espectacular en el mercado de las aplicaciones web.

2.1.1. Ventajas de Laravel

Para ir abriendo boca, te cuento algunos de los puntos fuertes que tiene Laravel:

- Sintaxis simple y elegante.
- Mapeo objeto-relacional (ORM): Eloquent. Ya veremos en qué consiste esto.
- Potente sistema de plantillas para vistas: Blade. También lo veremos con bastante profundidad.
- Reutiliza y moderniza componentes de Symfony.
- Es sencillo (esto es un decir) y potente.
- Uso creciente en la industria: es previsible que domine el mercado durante los próximos años.
- Comunidad de usuarios altamente especializada (buena relación señal/ruido, al menos de momento)

2.1.2. Inconvenientes de Laravel

Como no hay nada perfecto, es evidente que Laravel también tiene algunos defectillos. Entre ellos, estos:

- Instalación, configuración y despliegue complejos, incluso a través de servicios de virtualización.
- Curva de aprendizaje elevada.
- Se mueve según los intereses personales de su autor (es obra individual), con actualizaciones muy frecuentes y cambios caprichosos. Las actualizaciones pueden ser un quebradero de cabeza.
- Inestabilidad de varios de sus componentes: a menudo hay que recurrir a *fixes* o a componentes de terceros.

- Fuerte dependencia de la consola de comandos y de herramientas de terceros (composer, vagrant, npm...). Esto, por supuesto, solo es un inconveniente para las personas que tengan alergia a la consola de comandos.

2.2. Instalación de Laravel y creación de un proyecto

PASO 1: Instalar PHP y MySQL.

Instala PHP y MySQL. Como estás usando XAMPP, ya lo tienes instalado.

PASO 2: Instalar Git.

Para utilizar Composer es necesario tener previamente instalado y configurado Git.

Puedes descargarlo e instalarlo desde <https://git-scm.com/download/win>. Además, si no tenemos configurado el usuario de git nos presentará un error y nos ofrece las posibilidades siguientes:

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

PASO 3: Instalar Composer.

Instala Composer. Composer es una herramienta de gestión de dependencias para PHP que es necesaria para instalar Laravel. Para instalar Composer en Windows, sigue estos pasos:

Visita la [página de descarga de Composer](#) y descarga el archivo **Composer-Setup.exe**.

Windows Installer

The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run **Composer-Setup.exe** it will install the latest composer version whenever it is executed.

Ejecuta el archivo **Composer-Setup.exe** y sigue las instrucciones que aparecen en pantalla.

También se te pedirá que selecciones el modo de instalación. Asegúrate de elegir **Instalar para todos los usuarios (recomendado)**.

Asegúrate de seleccionar el ejecutable PHP correcto durante el proceso de instalación (normalmente se encuentra en la carpeta de instalación de XAMPP en `xampp/php/php.exe`).

Haz clic en **Siguiente** para seguir las instrucciones en pantalla y luego en **Instalar**.

Una vez completada la instalación, haz clic en **Finalizar**.

Para verificar que Composer se ha instalado correctamente, abre el símbolo del sistema y ejecuta el siguiente comando:

```
composer --version
```


Si la instalación se ha realizado correctamente, deberías ver la versión de Composer mostrada.

PASO 4: Crear proyecto

Puedes utilizar Composer, que ya está instalado, para instalar Laravel globalmente en tu sistema. Para ello, abre el símbolo del sistema y ejecuta el comando indicado:

```
composer create-project laravel/laravel mi-app
```

(Por supuesto, puedes cambiar “mi-app” por el nombre que tú quieras para la aplicación).

Esto descargará automáticamente todos los archivos relevantes de Laravel para crear un nuevo proyecto.

Si te saltan errores relacionados con “Zip compression”, cancela la creación del proyecto (Ctrl + C en la terminal), elimina el directorio que se está creando, instala 7zip y vuelve a probar a crear el proyecto.

PASO 5: Iniciar servidor web.

Con tu nuevo proyecto de aplicación creado, necesitarás iniciar un servidor. Para ello, dirígete a la carpeta donde se ha creado el proyecto:

```
cd mi-app
```

Y después inicia el servidor:

```
php artisan serve
```

PASO 6: Acceder al proyecto web.

Una vez iniciado el servidor, deberías poder acceder a tu proyecto de aplicación a través de tu navegador web. Para ello, abre tu navegador y ve a la siguiente dirección: <https://localhost:8000/>

Laravel usa un servidor web propio y por tanto no necesita que exista otro previo. En cambio, necesita algunos componentes ya instalados para el correcto funcionamiento. Estos componentes pueden ser muy variados, como por ejemplo el propio intérprete PHP.

Con esto, ya puedes empezar a desarrollar aplicaciones web utilizando Laravel en tu máquina Windows.

Actividad 5.0

Instala Laravel, crea un proyecto nuevo, inícialo y accede a él.

2.3. Arquitectura, convenciones y configuración de Laravel

Como cualquier framework, Laravel sigue una serie de convenciones para los identificadores, la estructura de directorios de la aplicación y la configuración.

Aunque no es imprescindible seguir esas convenciones, y puede resultar un poco enojoso las primeras veces (en plan: ¿pero por qué tengo que llamar a esa clase así, si quiero llamarla de otro modo?), a la larga el uso de convenciones ahorra muchísimo tiempo de desarrollo y, sobre todo, de depuración. ¿Cuántas veces no has estado depurando un error consistente en haber olvidado que el nombre de una tabla de la base de datos lo habías escrito en plural y no en singular, o al revés?

“**Convención sobre configuración**” es un lema de muchos frameworks. Quiere decir que, si seguimos las convenciones del framework, tanto si nos gustan como si no, nos ahorraremos mucho tiempo de discusión sobre la configuración y muchos quebraderos de cabeza posteriormente.

2.3.1. Estructura de directorios

Después de una instalación limpia de Laravel, nos encontraremos con una estructura típica de directorios que hay que respetar.

Los más importantes (al menos, para empezar) son estos:

composer.json: información para composer (ya sabes, el administrador de paquetes de PHP). Sirve para instalar todas las librerías de terceros que Laravel necesita para funcionar. Imprescindible para instalar la aplicación en otro servidor.

/app: el código de nuestra aplicación. Aquí están los **modelos**.

/config: configuración de la aplicación.

/app/Http: peticiones HTTP, incluyendo los **controladores**.

/database: migraciones y *seeders* de la base de datos. Las migraciones son muy importantes y les dedicaremos un apartado más adelante.

/plugins: pues eso.

/public: directorio de acceso público. Aquí Laravel generará todo lo que hay que mover al servidor para poner la web en producción. Puedes crear aquí dentro carpetas para colocar imágenes, scripts JS o archivos css.

/resources: Aquí dentro están las **vistas**. También el resto de *assets* (imágenes, css, js). A diferencia del directorio public, los archivos JS o css que coloquemos aquí estarán precompilados no serán accesibles vía web y, lo que es más importante, podemos crear archivos Less, Sass, JS y varios frameworks JS, y Laravel se encargará de compilarlos automáticamente y generar versiones minimizadas de nuestro CSS y nuestro JS.

De momento, vamos a colocar nuestro CSS y nuestro JS en la carpeta /public. Más adelante puedes trastear con /resources si lo deseas.

/storage: aquí Laravel guarda su memoria caché, información sobre las sesiones, vistas compiladas... No debes tocar esta carpeta para nada.

/vendors: librerías de terceros. Es **importante** añadir esta carpeta al `.gitignore` si vas a construir un repositorio git para tu aplicación Laravel, porque `/vendors` puede ocupar bastante espacio y no tiene sentido incluirla en tu proyecto. Si necesitas desplegar esta aplicación Laravel en otro servidor, basta con clonar el repositorio y ejecutar `composer update`. Eso *rellenará* la carpeta `/vendor` con las librerías más adecuadas para ese servidor.

2.3.2. Convenciones en Laravel

Laravel tiene un montón de convenciones que persiguen hacer más sencilla la configuración de la aplicación y evitar errores tontos que pueden consumir mucho tiempo de depuración.

Aunque la mayoría de estas convenciones no son obligatorias, es recomendable respetarlas para ahorrarte quebraderos de cabeza posteriores.

En primer lugar, usa solo **identificadores en inglés**. Mejor *User* que *Usuario*. Así, de paso, practicas un poco el idioma universal de la informática.

- **Modelos:** Los nombres de los modelos deben ser iguales a los de las tablas de la base de datos, pero en singular, en CamelCase y con mayúscula.

Ejemplo: *RegisteredUser*

- **Controladores:** Llámalos como los modelos, pero añadiendo la palabra “controller”.

Ejemplo: *RegisteredUserController*

- **Métodos:** Se nombran en camelCase y empezando con minúscula.

Ejemplo: *RegisteredUser::getAll()*

- **Atributos:** Se nombran en snake_case y empezando con minúscula.

Ejemplo: *RegisteredUser::first_name*

- **Variables:** Los identificadores deben ir en camelCase y empezando con minúscula. En plural si se trata de una colección y en singular si es un objeto individual o una variable simple.

Ejemplos: *bannedUsers* (colección, en plural), *articleContent* (objeto individual, en singular)

- **Tablas:** Se nombran en plural y en snake_case.

Ejemplo: *registered_users*.

- **Columnas de las tablas:** Se nombran en snake_case, sin referencia al nombre de la tabla.

Ejemplo: `first_name`.

- **Clave primaria:** La llamaremos *id*. Siempre. No le des más vueltas. *id* y punto. El tipo: *integer* y *auto-increment*.
- **Claves ajenas:** Se forman con el nombre de la tabla ajena en singular más la palabra *id*. Ejemplo: *article_id*.
- **Timestamps:** Laravel siempre crea marcas de tiempo para todo. Y siempre se llaman *created_at* y *updated_at*, de tipo *datetime*. Acostúmbrate a tenerlas en todas tus tablas.
- **Tablas pivot:** Son esas tablas horribles formadas como resultado de una relación N:N entre dos tablas maestras. Se nombran en *snake_case*, en plural y orden alfabético. Ejemplo: *articles_users* será el pivote entre las tablas *articles* y *users*.

2.3.3. Variables de entorno. El archivo `.env`

En el directorio raíz de toda aplicación hecha con Laravel existe un archivo muy especial llamado `.env`. Ojo, porque en los sistemas tipo Linux, todos los archivos cuyo nombre empieza por un punto están ocultos. Debes buscar la opción para mostrar los archivos ocultos de tu navegador de archivos si quieres verlo.

Este archivo contiene todas las variables de entorno que configuran la aplicación, como:

- `APP_ENV`: En esta variable se indica si la aplicación está en desarrollo o en producción
- `APP_DEBUG`: ¿Queremos mostrar los errores para depuración? Se pone a *true* durante el desarrollo y se cambia a *false* al pasar a producción.
- `APP_URL`: La URL base de la aplicación. Hay que modificarla cuando movemos la aplicación a otro servidor.
- `DB_CONNECTION`, `DB_HOST`, `DB_USERNAME`, etc. Es decir, la configuración de la conexión a la base de datos.

Una instalación limpia de Laravel vendrá con un archivo llamado ***example.env***, que contiene una plantilla para que puedas construir tu propio archivo `.env`. Simplemente, renombra el archivo `example.env` y asigna los valores correctos a las variables de entorno.

Cuidado: *el archivo .env NO debe sincronizarse con git* (o con el control de versiones que usemos) porque contiene información sensible. Asegúrate de incluirlo en tu *.gitignore*.

2.4. Artisan, la consola que mola

Artisan es una herramienta de consola, escrita en PHP, que viene con Laravel para **ayudarte a realizar tareas cotidianas en tu aplicación de forma automatizada**.

Algunas de esas tareas que automatiza Artisan son:

- Generar esqueletos de controladores y modelos.
- Crear migraciones de bases de datos (para manipular la estructura de las tablas, como enseguida veremos).
- Rellenar la BD con datos de prueba.
- Hacer el enrutamiento.
- Configurar la aplicación.
- Crear baterías de pruebas.
- Gestionar la caché de Laravel (una memoria intermedia generada por Laravel que hace que tu aplicación se cargue más rápido).

Y así otro montón de cosas.

No te preocupes si aún no entiendes qué significan algunas de estas cosas. Lo iremos viendo todo poco a poco. Lo importante ahora es que comprendas la tremenda utilidad de Artisan en el trabajo cotidiano con Laravel y que aprendas lo básico de su sintaxis.

2.4.1. Un ejemplo práctico: crear un controlador

Veamos cómo funciona Artisan con un ejemplo.

Supongamos que quieres crear un controlador. Para ello, existen dos maneras:

1. **A mano.** Nos vamos al directorio `/app/Http/Controllers` y creamos un archivo llamado, digamos, *HolaController.php*. Luego lo rellenamos con el esqueleto de un controlador vacío, copiando y pegando de otro controlador existente y eliminando todo lo que no nos haga falta.

Este método, obviamente, es lento, farragoso y resulta fácil que cometamos algún error.

2. **Con Artisan.** Simplemente, tecleamos este comando en el directorio raíz de nuestra aplicación:

```
$ php artisan make:controller HolaController
```

¡Hemos terminado! Artisan creará por nosotros el archivo `/app/Http/Controllers/HolaController.php` y lo rellenará con el esqueleto de un controlador vacío y sintácticamente correcto.

2.4.2. Comandos principales de Artisan

Una buena forma de empezar con Artisan es teclear este comando:

```
$ php artisan list
```

Ya te imaginas lo que hace, ¿verdad?

Por si acaso andas un poco despistado/a hoy, te lo cuento yo: Artisan te mostrará una lista con todos los comandos válidos en tu instalación de Laravel. Esto puede variar según tu versión de Laravel y los componentes que tengas instalados.

En un primer momento, estos son los comandos que más vas a utilizar (cuando te conviertas en un experto/a en Laravel, irán llegando otros por pura necesidad):

- `php artisan db:migrate`→ Para hacer migraciones (crear la estructura de nuestra base de datos).
- `php artisan db:seed`→ Para llenar de datos predefinidos nuestra base de datos.
- `php artisan make:migration`→ Crea una migración (para crear la estructura de la base de datos).
- `php artisan make:seeder`→ Crea un seeder (para rellenar con datos las tablas).
- `php artisan make:controller`→ Para crear un controlador.
- `php artisan make:model`→ Para crear un modelo.
- `php artisan route:list`→ Muestra todas las rutas definidas en el enrutador.

2.5. Creando la primera página

En esta sección vamos a crear nuestra primera aplicación funcional con Laravel: el tradicional “Hola mundo”.

Vale, ya sé que no es gran cosa, pero nos servirá para aprender la dinámica de trabajo con Laravel y para construir un esqueleto de aplicación sobre la que montar algo más grande más adelante.

Vamos a crear esta aplicación paso a paso. Tal vez te parezca un poco exagerado, pero recuerda que la curva de aprendizaje de Laravel es muy empinada al principio y se suaviza mucho después. Es decir: los primeros pasos son los que más cuestan.

PASO 1: Crea una clave segura.

Lo primero que debes hacer en cualquier aplicación Laravel es crear una clave segura para la aplicación.

Esto crea una clave de encriptación aleatoria que se guarda en el archivo de configuración de la aplicación (.env). Laravel utilizará esa clave para cifrar ciertas cosas (por ejemplo, las contraseñas de usuario).

Por ahora, no te preocupes de para qué sirve exactamente esta clave. Lo único importante es que Laravel no funcionará sin ella.

Así pues, lo primero que hay que hacer con cualquier aplicación Laravel nueva es ejecutar este comando:

```
$ php artisan key:generate
```

PASO 2: Capturar una ruta.

Vamos a empezar probando algo sencillo: capturar una ruta.

Edita el archivo `/routes/web.php` y añade esto:

```
Route::get('/hola', [HolaController::class, 'index']);
```

Esto indica al enrutador que, al recibir la ruta “hola” (<http://localhost:8000/hola>), se debe ejecutar el método `index()` del controlador `HolaController`.

También, tenemos que importar en este mismo archivo (`web.php`) el controlador. Para ello escribimos la siguiente línea al principio del todo:

```
use App\Http\Controllers\HolaController;
```

Pero el controlador *HolaController* no existe , así que tenemos que crearlo.

PASO 3: Crear controlador.

Vamos a crear el esqueleto del controlador con este comando de Artisan:

```
$ php artisan make:controller HolaController
```

Por último, edita el controlador (*/app/Http/Controllers/HolaController.php*) y añádele el método *index()*:

```
public function index() {  
    return "Hola, mundo";  
}
```

Comprueba que funciona en el navegador: <http://localhost:8000/hola>

PASO 4: Cargar una vista desde el controlador

¿Recuerdas que, en la arquitectura MVC, el controlador no debería producir ninguna salida HTML?

Pues bien, nuestro controlador la está produciendo, por lo que no está respetando las reglas del MVC.

Vamos a modificarlo para que el método *index()* del controlador no genere el “Hola, mundo”, sino que invoque a una **vista** que se encargue de ello.

Además, vamos a inyectar en la URL una variable con el nombre del usuario para mostrar cómo se capturan esos valores y cómo se pasan a las vistas en Laravel.

Comenzamos modificando el enrutador (*/routes/web.php*). Observa cómo se usan las llaves { y } para indicar la presencia de una variable en la URL:

```
Route::get('/hola/{nombre}', [HolaController::class, 'show']);
```

Ahora creamos un método *show()* en el controlador (*/app/Http/Controllers/HolaController.php*):

```
public function show($nombre) {  
    return view('holaVista', ['nombre' => $nombre]);  
}
```


Como ves, estamos invocando una vista llamada *holaVista* y le estamos pasando un array asociativo con los datos necesarios (el nombre del usuario, en este caso). Esa vista debe crearse en */resources/views/holaVista.blade.php* y puede tener este aspecto:

```
<body>
    Saludos, {{$nombre}}.
    ¿Le apetece una partidita de ajedrez?
</body>
```

Puedes probar esta nueva ruta cargando en el navegador una ruta como:

<http://localhost:8000/hola/ProfesorAlberto>

2.6. Enrutamiento

Llegados a este punto, ya debes haber creado tu primera aplicación Laravel (“Hola, mundo”) y habrás aprendido algo fundamental sobre el **enrutador**: cómo hacer que la solicitud de una URL termine ejecutando un método de un controlador.

En esta sección, profundizaremos en el enrutador, un componente fundamental de cualquier aplicación web.

2.6.1. ¿Qué es el enrutador?

El **enrutador** de Laravel es el componente que captura las URL solicitadas al servidor y las traduce a **invocaciones a métodos de los controladores**.

El enrutador es capaz, además, de **mapear fragmentos de la URL** a variables PHP que serán inyectadas como parámetros a los métodos del controlador.

Esto significa que, si le pides al servidor una ruta como esta:

`https://mi-servidor/user/delete/12`

...el enrutador puede “trocear” esa URL para extraer los segmentos (“user”, “delete” y “12”). Y tú puedes decidir qué hacer con cada uno de esos segmentos. Lo normal en este ejemplo sería que invocarás el método *delete()* del controlador *UserController*, y que ese método recibiera como parámetro el dato *12*, que será el id del usuario que se pretende borrar.

Y hay una última cosa que el enrutador puede hacer: **distinguir el verbo http** empleado para solicitar una página al servidor.

Es decir, para el enrutador no es lo mismo una solicitud enviada por GET que otra enviada por POST.

2.6.2. Enrutamiento básico

Hay varias **formas de generar una salida** HTML desde el enrutador (*/routes/web.php*). En este código de ejemplo vemos las cuatro formas básicas:

```
// Forma 1: generar la salida directamente en el enrutador, con un closure  
(función sin nombre)  
Route::get('/hola', function() {  
    return "Hola, mundo";  
});  
  
// Forma 2: llamar a una función de un controlador sin pasarle parámetros  
Route::get('/hola', [HolaController::class, 'show']);  
  
// Forma 3: llamar a una función de un controlador pasándole parámetros  
Route::get('/hola/{nombre}', [HolaController::class, 'show']);  
  
// Forma 4: llamar a una función de un controlador con un parámetro optativo  
Route::get('/hola/{nombre?}', [HolaController::class, 'show']);
```

La diferencia entre la forma 3 y la 4 es que, en la forma 3, la ruta debe llevar forzosamente un dato a continuación de “/hola” (algo como: <https://mi-servidor/hola/juan>). Si no lo lleva, el enrutador considerará que no se trata de esa ruta y seguirá buscando alguna ruta coincidente en el resto del archivo.

En cambio, en la forma 4, el dato final es optativo, así que el enrutador invocará el método *show()* del controlador tanto si ese dato aparece en la URL como si no lo hace. Posteriormente en el controlador, podemos comprobar si *\$nombre* es null.

2.6.3. Rutas con nombre

Es MUY recomendable **asignar un nombre a las rutas** en el enrutador. Esto hace que, más adelante, podamos cambiar la URL de los enlaces sin tener que modificar el código fuente de nuestras vistas.

El nombre se le asigna a una ruta añadiendo `->name('nombre')` al final:

```
Route::get('/contactame', [Controlador::class, 'contacto'])->name('contact');
```

En tu código fuente, debes referirte a esta ruta siempre con la expresión `route('contact')` (ya veremos exactamente cómo se hace esto), pero el usuario verá la dirección <https://servidor/contactame>.

En el futuro se puede cambiar la forma en la que lo ve el usuario. Por ejemplo, puedes cambiar `Route::get('/contactame'...)` por `Route::get('/acerca-`

de' . . .), pero no tendrás que modificar ni una línea de código más en tu aplicación, porque internamente esa ruta seguirá llamándose `route('contact')`.

2.7. Controladores

Los **controladores** son un elemento clave de la **arquitectura MVC**. En Laravel, los controladores funcionan exactamente igual que en las aplicaciones MVC hechas con PHP clásico.

Es decir: son los puntos de entrada a la aplicación desde el enrutador.

Los controladores deberían permanecer lo más sencillos posible: nada de accesos a la base de datos ni de generación de HTML. Esas acciones se derivarán a los modelos y las vistas. El controlador es un organizador del flujo de la aplicación: decide qué componente tiene que trabajar y el orden en el que lo debe hacer.

En esta sección, vamos a aprender a construir controladores con Laravel. Vas a ver que es algo extremadamente sencillo.

2.7.1. Cuatro cosas importantes sobre los controladores en Laravel

1. Los **controladores** en Laravel siempre **heredan de la clase *Controller*** o de una subclase de *Controller*.

```
class MiControlador extends Controller {  
    ...  
}
```

2. Su nombre debería escribirse en singular, CamelCase y terminando en la palabra *Controller*. Por ejemplo: *UserController*, *LoginController*, *ArticleController*...
3. **Cada método del controlador debe terminar en un *return***. Lo que el método devuelva será convertido automáticamente en una *HTTP response 200*, es decir, en una respuesta válida http, excepto si es un array, en cuyo caso Laravel lo convertirá automáticamente en JSON (perfecto para responder a peticiones Ajax, como veremos más adelante).
4. Si lo necesitas, puedes actuar sobre la *HTTP response* devolviendo algo como esto:

```
return response('contenido', 201)  
    ->header('mi-cabecera', 'contenido')  
    ->header('otra-cabecera', 'más-contenido')  
    ->cookie('mi-cookie', 'valor');  
(etc)
```

(Pero no nos engañemos: no lo necesitarás hacer demasiado a menudo).

2.7.2. Crear un controlador

Los controladores se pueden crear a mano: vas al directorio `/app/Http/Controllers`, creas allí un archivo vacío y empiezas a escribir código como una fiera.

Pero nadie lo hace así porque **Artisan** ya crea el esqueleto del archivo por ti. Así que es mejor que vayas a lo práctico: abre una consola en tu servidor web y ponte a escribir.

```
$ php artisan make:controller UserController
```

Después, ya puedes abrir el archivo `/app/Http/Controller/UserController.php` y ponerte a modificarlo.

2.7.3. Redirecciones

En el controlador, podemos redireccionar a una URL de nuestra aplicación o externa.

- De nuestra aplicación, usando `redirect()->route()`:

```
return redirect()->route('nombre_ruta');
```

- También podemos incluir parámetros:

```
return redirect()->route('nombre_ruta', ['id' => 1]);
```

- Cualquier URL, usando `redirect()->away()`:

```
return redirect()->away('https://www.google.es');
```

Actividad 5.1

Crea un controlador llamado `CalculadoraController`.

Prueba a crear una ruta con el siguiente formato: `/suma/15/30` que muestre el resultado de la suma de los dos números. Por ahora no uses vistas. Genera el resultado en el propio controlador.

Actividad 5.2

Crea una nueva ruta similar a la de la actividad anterior pero haciendo que los dos parámetros sean opcionales. Haz que el valor por defecto de ambos números sea 2. A continuación se presentan algunos ejemplos de la salida esperada:

- /suma/5/6 mostrará como resultado 11

- /suma/5 mostrará como resultado 7

- /suma mostrará como resultado 4

Actividad 5.3

Crea un controlador llamado SorteoController.

Prueba a crear una ruta con el siguiente formato: /sorteo/numero/25 que muestre el resultado de un número aleatorio entre 0 y el número introducido en la URL. El método del controlador deberá llamarse "numero". Si no se indica ningún número en la URL, haz que por defecto sea 100.

Actividad 5.4

Modifica SorteoController::numero para que, si no recibimos un número, nos redirija a esa misma ruta pero con un máximo de 50.

2.8. Vistas y plantillas. Blade

Los controladores se encargan de recibir la petición y procesarla, y para mostrar la información al usuario generan la respuesta ayudándose de plantillas. De esta manera estamos separando la información y su procesamiento (**Modelo y Controlador**) de la forma en la que se le muestra al usuario (**Vista**).

Blade es un poderoso y sencillo sub-lenguaje que incorpora Laravel que nos permitirá **generar plantillas de vistas** para minimizar el código que necesitaremos para nuestras vistas.

Las plantillas Blade admiten condiciones y bucles para operar con las variables PHP, de modo que la misma plantilla se comporta de forma diferente con diferentes conjuntos de datos. ¡Y se acabó la pesadilla de abrir y cerrar comillas en las instrucciones `echo`!

Con Blade, tampoco será necesario abrir y cerrar php (`<?php . . . ?>`) para operar con las variables del servidor y generar la salida.

El código escrito con Blade no solo es más limpio y fácil de depurar que con PHP básico, sino también más seguro, porque Blade impide cualquier ataque con XSS.

Pero no es todo. Además, unas plantillas pueden heredar de otras para reutilizar mejor el código.

Así que Blade tiene un montón de ventajas y ningún inconveniente. Bueno, sí que tiene un inconveniente: que tienes que aprender a usarlo. Pero es tan sencillo que, cuando hayas trabajado con Blade un par de semanas, te preguntarás cómo habías podido sobrevivir hasta ahora sin él.

2.8.1 Master layout. La plantilla padre

Layout es una forma genérica de referirse al diseño de capas del interfaz de usuario de una aplicación. Es decir, el *layout* de una aplicación es el diseño de los elementos de su interfaz de usuario.



El **master layout** es el diseño maestro del que derivan/heredan todas las vistas. Las aplicaciones web suelen tener un *master layout* muy definido, y todas las vistas de la aplicación lo respetan.

Ese *master layout* suele almacenarse, por convenio, en un archivo llamado `/resources/views/layouts/master.blade.php`

Vamos a construir un *master layout* de ejemplo. Será muy simple, pero más adelante puedes complicarlo todo lo que quieras, y cada modificación afectará automáticamente a todas las vistas de tu aplicación.

```
<html>
  <head>
    <title>@yield('title', 'Título por defecto')</title>
  </head>
  <body>
    @section('sidebar')
      Este es mi master sidebar.
    @show

    <div class="container">
      @yield('content')
    </div>
  </body>
</html>
```

Aquí nos encontramos con varias **directivas** importantes de Blade:

- **@yield('title')** → Esto define una sección llamada 'title', que en el master layout tiene valor por defecto "Título por defecto". Si las vistas que hereden del master layout definen esta misma sección, se utilizará el valor de la vista hija. Si no, se utilizará el valor por defecto.
- **@section('sidebar')** → Esto define una sección llamada 'sidebar' que NO está vacía (contiene el texto "Este es mi master sidebar", pero podría contener cualquier otra cosa). Funciona igual que @yield, solo que permite tener contenido por defecto más extenso.
- **@show** → Hace que el contenido de la sección se muestre en ese punto del documento.
- **@yield('content')** → Una sección que está vacía en el master layout y cuyo contenido tendrá que definirse en las vistas que hereden de esta. A diferencia del primer @yield, este no tiene ningún valor por defecto.

2.8.2 Vistas que heredan del master layout. Las plantillas hijas

Después de crear nuestro *master layout*, lo siguiente es empezar a construir vistas que lo utilicen.

Es decir, que **hereden** de él todos sus componentes y añadan nuevos elementos, respetando la configuración básica establecida por el *master layout*.

El aspecto de una vista que herede del *master layout* de nuestro ejemplo anterior debería ser este:

```
@extends('layouts.master')

@section('title', 'Título de la página')

@section('sidebar')
    @parent
    <p>Esto se añadirá al master sidebar.</p>
@endsection

@section('content')
    <p>Aquí va el contenido de mi página.</p>
@endsection
```

De nuevo tenemos varias *directivas* de Blade muy importantes:

- **@extends('layouts.master')** → Esto indica que esta vista hereda del master layout que se encuentra dentro de */resources/views/layouts/master.blade.php*.
- **@section('title', 'Título de la página')** → Aquí se define el contenido de la sección 'title' que en el master layout tenía como valor por defecto "Título por defecto".
- **@section('sidebar') @parent** → Gracias al usar @parent, el contenido se añade a la sección 'sidebar', que en el master layout no estaba vacía. Si no se hubiera usado @parent, el contenido se hubiera sobrescrito.
- **@section('content')** → Aquí se define el contenido de la sección 'content' que en el master layout estaba vacía.
- **@endsection** → Se emplea cuando el contenido de una sección ocupa varias líneas de código y no es posible escribirlo dentro de la propia directiva.

2.8.3 Cómo pasar variables a las vistas desde el controlador

Las **variables** a las vistas deben enviarse desde el controlador **empaquetadas en un array asociativo**.

Hay varias formas de lograr esto, todas ellas correctas. Puedes emplear la que te resulte más cómoda o más elegante:

```
// Forma 1: crear el array antes de llamar a la vista
$data['variable1'] = 'valor';
$data['variable2'] = 'valor';
return view('vista', $data);

// Forma 2: crear un array con el constructor array()
return view('vista', array('variable1'=>'valor', 'variable2'=>'valor'));

// Forma 3: crear un array con corchetes []
return view('vista', ['variable1'=>'valor', 'variable2'=>'valor']);

// Forma 4: usar el método with()
return view('vista')->with(['variable1'=>'valor', 'variable2'=>'valor']);

// Forma 5: usar la función compact()
return view('vista', compact('variable1', 'variable2'));
```

2.8.4 Cómo usar las variables desde el controlador en las vistas

Vale, en el controlador ya hemos enviado algunas variables a las vistas. Ahora, ¿cómo las usamos?

La vista no recibe el array con las variables, sino **las variables ya extraídas del array**. Esto significa que, si le pasas a una vista un par de variables como `['variable1'=>'valor', 'variable2'=>'valor']`, en la vista existirán dos variables llamadas `$variable1` y `$variable2`, y no encontrarás ni rastro del array que las contenía.

Veámoslo con un ejemplo. Supongamos que tenemos este método en un controlador:

```
function show() {
    return view('page', array('name' => 'Manolo Escobar'));
}
```

En la vista `/resources/views/page.blade.php` tendrás disponible una variable `$name`, y la puedes usar de este modo:

```
@extends('layouts.master')

@section('title', 'Page Title')
@section('sidebar')
    @parent
    <p>Esto se añadirá al sidebar del master layout.</p>
@endsection

@section('content')
    <h2>{{ $name }}</h2>
    <p>Este es el contenido de mi página.</p>
@endsection
```

Observa que hemos respetado el *master layout* de ejemplo que creamos más arriba. Observa también cómo se hace el *echo* de la variable: nada de abrir y cerrar PHP (`<?php ... ?>`) ni de abrir y cerrar comillas: simplemente, se encierra la variable entre llaves dobles (`{{ $variable }}`), y Laravel se encargará de traducir eso al *echo* correspondiente.

La vista queda de ese modo mucho más limpia y, por lo tanto, es más fácil de escribir, leer y depurar.

2.8.5 Condiciones en las vistas

Blade admite **expresiones condicionales** para modificar el aspecto de una vista dependiendo del valor de una variable o del estado de la aplicación.

Por supuesto, eso también puede hacerse sin Blade: solo hay que usar un *if* de PHP. Pero con Blade es más fácil y limpio porque, nuevamente, no tendremos que andar abriendo y cerrando PHP ni peleándonos con las comillas del *echo*.

En el siguiente ejemplo, el contenido de la vista depende de la variable `$day` enviada desde el controlador. Observa como el *if* de PHP se sustituye por la directiva `@if` de Blade.

Controlador:

```
function show() {
    return view('page', array('name' => 'Manolo', 'day' => 'Viernes'));
}
```

Vista `/resources/views/page.blade.php`:

```
@extends('layouts.master')

@section('title', 'Page Title')
@section('sidebar')
    @parent
    <p>Esto se añadirá al sidebar del master layout.</p>
@endsection

@section('content')
    <h2>{{ $name }}</h2>
    <p>Este es el contenido de mi página.</p>
    @if ($day == 'Viernes')
        <p>Hoy me roban el carro</p>
    @else
        <p>Hoy me dedico a cantar</p>
    @endif
@endsection
```

2.8.6 Bucles en las vistas

Igual que existen condiciones, Blade también permite hacer **bucles** para simplificar la generación de vistas.

Por ejemplo, en la siguiente vista recorreremos un array llamado `$bebidas` y mostramos su contenido.

Controlador:

```
function show() {
    $bebidas = array('Cerveza', 'Agua', 'Zumo');
    return view('page', array('name' => 'Manolo', 'day' => 'Lunes',
                             'bebidas' => $bebidas));
}
```

Vista `/resources/views/page.blade.php`:

```
@extends('layouts.master')

@section('title', 'Page Title')
@section('sidebar')
    @extends
    <p>Esto se añadirá al sidebar del master layout.</p>
@endsection

@section('content')
    <h2>{{$name}}</h2>
    <p>Este es el contenido de mi página.</p>
    @if ($day == 'Viernes')
        <p>Hoy me roban el carro</p>
    @else
        <p>Hoy me dedico a cantar</p>
    @endif
    <h2>Estas son las bebidas disponibles:</h2>
    @foreach ($bebidas as $bebida)
        {{$bebida}} <br>
    @endforeach
@endsection
```

Observa que, nuevamente, esto mismo podría hacerse con PHP clásico, pero con Blade el código queda mucho más limpio y resulta menos propenso a errores.

2.8.7 Invocación de funciones PHP

Desde una vista Blade se puede **invocar cualquier función de PHP** sin necesidad de abrirlo y cerrarlo (ya sabes: nada de `<?php ... ?>` en las vistas).

Por ejemplo, si necesitamos usar la función `date()`, la invocamos así:

```
<h2>Ejemplo de llamada a función estándar de PHP</h2>
<p>La fecha del servidor es: {{date(' D M, Y')}}</p>
```

Esto no solo es más limpio, sino también más seguro, porque Blade filtra posibles ataques por XSS.

(Por cierto: si alguna vez necesitas saltarte la protección contra XSS, puedes usar `{!! ... !!}` en lugar de `{{ ... }}`).

2.8.8 Otras directivas Blade

Blade dispone de multitud de directivas para hacer prácticamente cualquier cosa en la vista sin tener que recurrir a PHP. En esta lista mostramos solo las directivas más habituales.

- **@section** → Marca el inicio de una sección de contenido.
- **@endsection** → Marca el final de una sección.
- **@show** → Marca el final de una sección y la muestra.
- **@yield('section')** → Muestra el contenido de una sección (si existe, claro).
- **@extends('view')** → Hereda de una plantilla padre.
- **@parent** → Muestra el contenido de la sección del mismo nombre en la plantilla padre.
- **@include('view')** → Incluye una subvista.
- **@if - @else - @endif** → Condicional.
- **@for - @endfor** → Bucle for clásico.
- **@foreach - @endforeach** → Bucle foreach.
- **@forelse - @empty - @endforelse** → Como foreach, pero con tratamiento de arrays que vienen vacíos.
- **@break - @continue** → Para usar en los bucles.
- **@switch - @case - @break** → Condicional múltiple.
- **@isset(\$variable) - @endisset** → Comprueba si la variable existe.
- **@auth - @endauth** → La sección solo se muestra si hay un usuario autenticado en la aplicación.
- **@php - @endphp** → Para añadir código PHP plano. Usar con moderación.
- **{{- Comment -}}** → Comentarios (no serán renderizados).
- **{{ \$variable }}** → Equivalente a echo \$variable, pero no es necesario abrir y cerrar PHP, y nos protegemos contra inyección de JS.
- **{{ \$variable ?? texto-por-defecto }}** → Como el anterior, pero chequea si la variable existe. Si existe, muestra su valor. Si no, muestra texto-por-defecto.
- **\$looop** → Es una variable muy útil para usar en y/o depurar un foreach. Nos dice si estamos en el primer elemento, en el último, cuántos loops llevamos, etc.

Actividad 5.5

Crea una vista padre (master layout) llamado “vistaNumeros” que contenga las etiquetas básicas de html (html, head y body). Dentro del body crea una etiqueta de título (<h1>) vacía.

Haz que lo siguiente se pueda editar por las vistas hijas:

- *Título de la página (<title>).*
- *Título dentro de <h1>.*
- *Contenido dentro del <body> (después del <h1>, claro).*

Actividad 5.6

Modifica la Actividad 5.2 para hacer uso de vistas en lugar de generar el html en el propio controlador. Para ello:

1. Crea una vista llamada “sumaCalculadora” que herede de la vista master creada en la Actividad 5.5.
2. Desde el controlador, pásale a la vista: primer número, segundo número y resultado de la suma.
3. En la vista, muestra estos datos.

Ten en cuenta que:

- En el título de la página (<title>) deberá mostrarse “Suma de números”.
- En el título <h1> deberá mostrarse “Suma de números”
- En el contenido de <body> deberá mostrarse la operación realizada con los dos números y el resultado final.

Actividad 5.7

Modifica la Actividad 5.3 para hacer uso de vistas en lugar de generar el html en el propio controlador. Para ello:

1. Crea una vista llamada “sorteoNumeros” que herede de la vista master creada en la Actividad 5.5.
2. Desde el controlador, pásale a la vista: número máximo y resultado del sorteo.
3. En la vista, muestra estos datos.

Ten en cuenta que:

- En el título de la página (<title>) deberá mostrarse “Sorteo de números”.
- En el título <h1> deberá mostrarse “Sorteo de números”
- En el contenido de <body> deberá mostrarse el número máximo con el que se ha realizado el sorteo y el número resultante.

2.8.9 Tratamiento de formularios con Blade

Blade también facilita un montón el **tratamiento de los formularios**. Y, por supuesto, filtra por nosotros cualquier código malicioso que traten de colarnos a través de ellos.

Vamos a verlo con un ejemplo.

Imaginemos una vista llamada *form.blade.php* con este formulario (fíjate cómo indicamos la ruta en el *action*):

```
<form method="POST" action="{{ route('procesarForm') }}">
    @csrf    <!-- Para evitar ataques CSRF -->
    <input type="email" name="email"><br>
    <input type="text" name="asunto"><br>
    <textarea name="contenido"></textarea><br>
    <button type="submit">Enviar</button>
</form>
```

Fíjate en la directiva **@csrf**. Todos los formularios deberían llevarla. Impide los ataques por CSRF, incrustado un token aleatorio en el formulario que Laravel se encargará de comprobar cuando los datos del formulario regresen al servidor. No te preocupes si no entiendes bien como funciona, por Laravel se encarga de hacerlo todo. Tú solo tienes que colocar la directiva **@csrf** en todos tus formularios.

Por supuesto, para mostrar este formulario deberíamos crear un método en el controlador que muestre su vista, así como definir la ruta.

La definición de la ruta en *web.php*:

```
Route::get('/formulario', [HolaController::class, 'form']);
```

La declaración del método en el controlador *HolaController.php*:

```
function form() {
    return view('form');
}
```

Ahora, si accedemos a la ruta */formulario*, podremos ver el formulario. Fíjate que el formulario tiene un *action* que apunta a una ruta llamada *procesarForm*.

En el enrutador, definiremos la ruta “*procesarForm*” del action del formulario:

```
Route::post('/procesado', [HolaController::class, 'procesar']->name('procesarForm'));
```

Fíjate como ahora el método es *post* en lugar de *get*.

Y, en el controlador, creamos el método *procesar()*. Observa cómo recuperamos los datos del formulario a través de la variable *\$r*:

```
public function procesar(Request $r) {  
    $email = $r->get("email");  
    $asunto = $r->get("asunto");  
    ...etc...  
}
```

2.8.10 Validación de formularios con Blade

Otra cosa que Blade facilita muchísimo es la **validación de formularios en el lado del servidor**.

Vamos a retomar **la vista** del ejemplo anterior y a modificar el formulario para que se valide en el servidor. Tendríamos que añadirle algunas cositas:

```
<form method="POST" action="{{ route('procesarForm') }}">  
    @if ($errors->any())  
        @foreach ($errors->all() as $error)  
            {{ $error }}<br>  
        @endforeach  
    @endif  
    <input type="email" name="email"><br>  
    ...resto del formulario igual...  
</form>
```

Fíjate en cómo mostramos los errores -si los hay- con un bucle. El objeto *\$errors* (disponible en todas las vistas) tiene muchos más métodos útiles que puedes consultar en la documentación de Laravel.

Y el **controlador** quedaría así. Observa cómo definimos varias **reglas de validación** para campos del formulario:

```
public function store(Request $r) {  
    $r->validate([  
        'email' => 'required|email',  
        'asunto' => 'required'  
    ]);  
    // A partir de aquí, se procesa el formulario igual que antes
```

Puedes consultar todas las reglas de validación admitidas en <https://laravel.com/docs/validation>.

Cuando vuelve a cargarse el formulario que contenía un error, suele ser apropiado hacerlo con los datos que el usuario ya había tecleado. A esto se le llama **“repopular” el formulario**, y con Blade se hace así (observa el atributo *value* del campo *email*):

```
<form method="POST" action="{{ route('procesarForm') }}">  
    @if ($errors->any())  
        @foreach ($errors->all() as $error)  
            {{ $error }}<br>  
        @endforeach  
    @endif  
    <input type="email" name="email" value="{{ old('email') }}"><br>  
    ...resto del formulario igual...  
</form>
```

2.8.11 Añadir recursos (imágenes, css, js...) a nuestras vistas

Todos los recursos, ya sean imágenes, css y archivos javascripts que solemos referenciar en HTML deben colocarse en la carpeta *public*. Una vez colocados, podemos generar sus rutas automáticamente utilizando la función **asset** en la vista Blade.

Ejemplo de como mostrar una imagen llamada *imagen.png* que se ha introducido dentro de la carpeta *public*:

```

```

Actividad 5.8

Modifica la Actividad 5.6 para hacer uso de un formulario para introducir los dos números que se quieran sumar (en lugar de escribir los números en la url).

Ten en cuenta que deberás crear dos rutas y dos métodos en el controlador: el primero para mostrar el formulario y el segundo para mostrar el resultado de la suma.

Debes validar en el lado servidor que los dos números introducidos sean enteros.

Actividad 5.9

Modifica la Actividad 5.7 para hacer uso de un formulario para introducir el número máximo del que se quiere obtener un número aleatorio (en lugar de escribirlo en la url).

Debes validar en el lado servidor que el número introducido sea entero y que además sea positivo.

2.9. Modelo. ORM Eloquent

Eloquent uno de los componentes de Laravel que permiten al desarrollador manipular los datos de la BD sin rebajarse a escribir sucio SQL. Y lo consigue mediante un mecanismo simple y elegante: el mapeo objeto-relacional.

En esta sección vamos a ver qué es eso del mapeo y cómo sacarle a Eloquent el máximo partido para que nuestros modelos se programen prácticamente solos.

2.9.1 ¿Qué es el mapeo objeto-relacional?

Eloquent no es más que una librería incluida con Laravel que utiliza un patrón de software llamado **ORM (Object-Relational Mapping)** para abstraer aún más el acceso a la base de datos, de manera que no tengamos que escribir y depurar SQL.

Mapear los objetos de nuestra aplicación con una BD relacional significa que Eloquent convierte los registros de tus tablas en objetos de tu aplicación para que los manipules con mayor facilidad.

Sí, lo has entendido bien: podrás manejar los datos de tu base de datos como si fueran objetos de tu aplicación. Y, cuando los modifiques, borras o crees, se ejecutará el código SQL necesario (sin que tú te enteres) para traducir esas operaciones en sentencias para la base de datos.

Te lo muestro con un ejemplo.

Imagina que tenemos una tabla de artículos de un blog. La llamaremos *Articles*, y tendrá 3 campos: el *id* (entero), el *title* (cadena de caracteres) y el *body* (cadena de caracteres). Con Eloquent, usar esa tabla es tan fácil como hacer algo así:

```
$art = Article::find('7');           // Buscamos un artículo por su id
echo $art->title;                     // Ahora podemos acceder a los campos de ese artículo
$art->body = "Texto del cuerpo";      // O también podemos modificar los campos del artículo
$art->save();                         // Si hacemos save(), los cambios se guardan en la BD
```

2.9.2 Configurando los parámetros de la BD

Antes de empezar, para que cualquier operación sobre la base de datos funcione, debes tener bien configuradas estas variables de entorno del archivo `.env` de Laravel:

```
DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=mi-base-de-datos
DB_USERNAME=mi-usuario-de-BD
DB_PASSWORD=mi-password-de-BD
```

2.9.3 Crear el modelo

Tienes que **crear un modelo**. ¿Qué te creías? Pero con Artisan es así de fácil:

```
$ php artisan make:model <Mi-modelo> -m
```

Por ejemplo:

```
$ php artisan make:model Article -m
```

El modelo se creará en `/app/models/Article.php` (¡cuidado! En versiones más antiguas de Laravel, el modelo se creará en `/app/Article.php`)

Ya tienes tu modelo. Si no puedes contener tu curiosidad insaciable y lo abres, verás un archivo bastante decepcionante con este aspecto:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Article extends Model
{
    use HasFactory;
}
```

¿Y eso es todo?

Pues sí, eso es todo. La gracia está en ese “*extends Model*” de la definición de la clase: aunque tu modelo `Article` parezca vacío, en realidad acaba de heredar un mogollón de características útiles de la clase `Model`. Entre otras, todos los métodos de `Eloquent`.

2.9.4 Añadir campos a la tabla. Migraciones

Las **migraciones** constituyen una especie de control de versiones para la base de datos de la aplicación. Permiten crear y modificar tablas de la BD con independencia del SGBD que estemos usando.

La primera cosa para las que sirven las migraciones es para **crear las tablas de tu aplicación**. Olvídate de exportar e importar a mano el archivo SQL de tu base de datos. Eso lo hacían los cromañones.

Antes, cuando usaste el comando para crear el modelo *Article*, utilizaste el parámetro `-m`. Gracias a ese parámetro, se creó una migración automáticamente en `/database/migrations`. Tendrá un nombre parecido a `2025_01_12_143029_create_articles_table.php`.

Si editas ese fichero, verás dos métodos:

- **up()** → se ejecuta cuando se lanza la migración. Se encarga de crear la tabla en la base de datos.
- **down()** → se ejecuta cuando se cancela la migración. Se encarga de eliminar la tabla de la base de datos.

Nosotros nos fijaremos en el método `up()`, que tendrá un aspecto tal que así:

```
public function up() {  
    Schema::create('articles', function (Blueprint $table) {  
        // La siguiente línea crea la columna id  
        $table->id();  
        // La siguiente línea crea campos created_at y updated_at  
        $table->timestamps();  
    });  
}
```

Vamos a añadir varias columnas relacionadas con la entidad *Article*.

```
public function up() {  
    Schema::create('articles', function (Blueprint $table) {  
        $table->id();  
        // Columna del título. 75 caracteres como máximo  
        $table->string('title', 75);  
        // Columna del texto del artículo. Debe ser único  
        $table->string('content')->unique();  
        // Columna de la fecha de publicación  
        $table->date('publish_date');  
        // Columna de la categoría. Puede estar vacío  
        $table->text('category')->nullable();  
        // Número de visitas del artículo  
        $table->integer('views');  
        $table->timestamps();  
    });  
}
```

Una vez tengamos el archivo de la migración terminado, tenemos que lanzar la migración con el siguiente comando:

```
$ php artisan migrate
```

¡Y listo! Ya tendrás creada la tabla *articles* en la base de datos.

Si por algún motivo quieres modificar la tabla porque te has equivocado o porque quieres añadir una nueva columna, tendrás que deshacer la creación de la tabla con el comando:

```
$ php artisan migrate:rollback
```

Después deberás modificar el archivo de migración y por último tendrás que volver a lanzar la migración.

2.9.5 Consultas con Eloquent

Solo con construir el modelo (y asignarlo a la tabla adecuada) ya tenemos detrás a Eloquent haciendo el mapeo objeto-relacional.

Ahora podemos ir a nuestro controlador (que, por respetar las convenciones de Laravel, debería llamarse *ArticlesController*) y **lanzar consultas** con expresiones como estas:

```
// Devuelve todos los artículos
$articlesList = Article::all();
// Devuelve el artículo con ese $id
$myArticle = Article::find($id);
// Select con where
$articlesList = Article::where('id', '>', 100)->get();
// Select con límite de 10 filas devueltas
$articlesList = Article::where('id', '>', 100)->take(10)->get();
// Select con dos condiciones (AND)
$articlesList = Article::where('id', '>', 100)->where('id', '<', 200)->get();
// Select con dos condiciones (OR)
$articlesList = Article::where('id', '>', 100)->orWhere('id', '=', 50)->get();
// Devuelve el último id asignado
$maxId = Article::max('id');
```

A lo mejor hay algo que te está chirriando. Si escribimos cosas como estas en el controlador, ¿no significa eso que estamos lanzando consultas desde el controlador? Y eso está prohibidísimo en la arquitectura MVC. ¿Es que Laravel no respeta la arquitectura MVC?

Hay quien dice que sí, hay quien dice que no y hay quien dice que psé, psé.

En realidad, no estamos escribiendo SQL en el controlador, ni estamos accediendo a la base de datos de ninguna manera explícita. Cuando lanzamos un `Article::find($id)` desde el controlador, por ejemplo, no sabemos qué ocurre por debajo ni de dónde se extrae la información: nos limitamos a acceder a una de nuestras clases y obtener de ella un objeto de tipo *Article*.

2.9.6 Inserciones y borrados con Eloquent

También podemos usar Eloquent para **insertar** un nuevo artículo desde nuestro controlador:

```
$art = new Article();  
$art->title = 'Los Marcheneros invaden Nueva York';  
$art->content = 'Bla, bla, bla';  
$art->publish_date = '2025-01-10';  
$art->category = 'Drama';  
$art->views = 333;  
$art->save();
```

Y, por supuesto, también podemos **modificar y borrar** artículos de la base de datos:

```
// MODIFICAR  
$art = Article::find(18);  
$art->body = 'Nuevo cuerpo';  
$art->save();  
  
// BORRAR  
$art = Article::find(13);  
$art->delete();
```

2.9.7 Inserción de entidades desde un formulario

Si los datos del artículo vienen de un formulario, tenemos la posibilidad de crear un objeto `Article` directamente sin tener que rellenar sus atributos uno a uno.

Para ello, primero debemos modificar el modelo añadiendo el atributo `$fillable` con los nombres de los campos de la entidad:

```
class Article extends Model
{
    use HasFactory;

    // Campos de la tabla en los que se permite la ASIGNACIÓN MASIVA
    // (útil para los formularios por ejemplo)
    protected $fillable = ['title', 'content', 'publish_date', 'category', 'views'];
}
```

A la hora de procesar un formulario, fíjate en lo alucinantemente fácil que es recoger todos esos datos, crear un objeto `Article` con ellos y guardar el artículo en la BD:

```
public function store(Request $request) {
    Article::create($request->all()); // Esto es una ASIGNACIÓN MASIVA
    return view('la-vista-que-sea');
}
```

2.9.8 Lista de los métodos más útiles de Eloquent

Hemos visto en los últimos ejemplos **algunos métodos de Eloquent** por separado. Te los reúno en esta sección para que los puedas consultar cuando lo necesites.

Aviso: no están todos, solo los de uso más habitual. Si quieres una lista completa, ya sabes: acude a la [documentación oficial](#).

- **all()** → Recupera todos los registros de una tabla.
- **where(“campo”, valor)** → Aplica cláusula *where*.
- **orderBy(“campo”, “asc|desc”)** → Aplica cláusula *order by*.
- **get()** → Recupera registros seleccionados. Se suele usar con *where* y/o *order by*.
- **first()** → Recupera el primer registro.
- **latest()** → Recupera el último registro.
- **find(valor)** → Busca registros con ese valor en el campo *id*.
- **findOrFail(valor)** → Lanza un error 404 si no encuentra el registro.

- **count(), max(), min()...** → Utiliza funciones de agregado de SQL.
- **save()** → Inserta o actualiza registros.
- **update()** → Actualiza registros.
- **delete()** → Elimina registros.

Actividad 5.10

Crea una entidad Producto con los atributos: nombre (string), descripción (string), precio (float) y stock (integer).

Recuerda que debes crear su modelo, modificar la migración y lanzar la migración.

Actividad 5.11

Crea un controlador llamado ProductoController.

Crea un formulario que recoja los datos de un producto y lo inserte en la base de datos. Tanto la ruta para mostrar el formulario como para procesarlo será /producto.

Agrega 4 productos.

Actividad 5.12

*Crea un método en el controlador ProductoController llamado mostrarProductos() que se ejecutará al entrar al enlace /productos. Este método deberá obtener todos los productos de la base de datos y pasarlos a la vista, donde se mostrarán en forma de tabla. La tabla tendrá dos columnas extras al final llamadas “Borrar” y “Actualizar”. La celda de borrar contendrá un enlace a /producto/borrar/**id** y la de actualizar un enlace a /producto/modificar/**id**.*

Actividad 5.13

*Crea un método en el controlador ProductoController llamado borrarProducto() en el enlace /producto/borrar/**id**. Este método deberá borrar de la base de datos el producto con el id recibido a través de la url. Una vez borrado, se redirigirá a la ruta donde se muestran los productos (Actividad 5.12).*

Actividad 5.14

*Crea un método en el controlador ProductoController llamado actualizarProducto() en el enlace /producto/modificar/**id**. Este método deberá mostrar un formulario con los datos prerellenos del producto indicado con el id a través de la url. Al enviar el formulario, se actualizará el producto en la base de datos con los nuevos datos modificados en el formulario.*